



P00 - Java

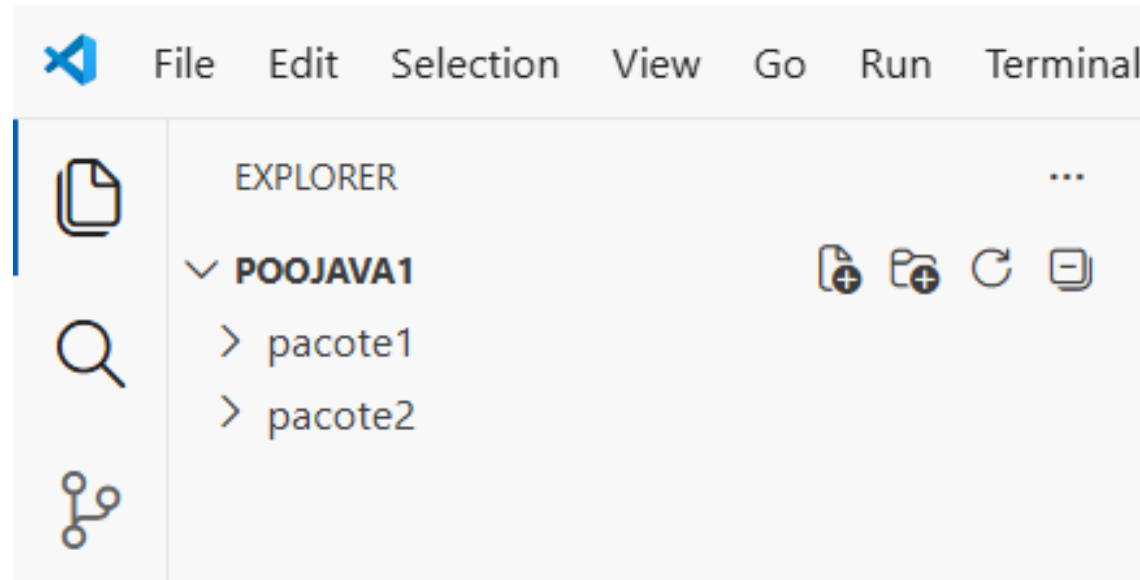
Introdução

PAULO R T CÂNDIDO

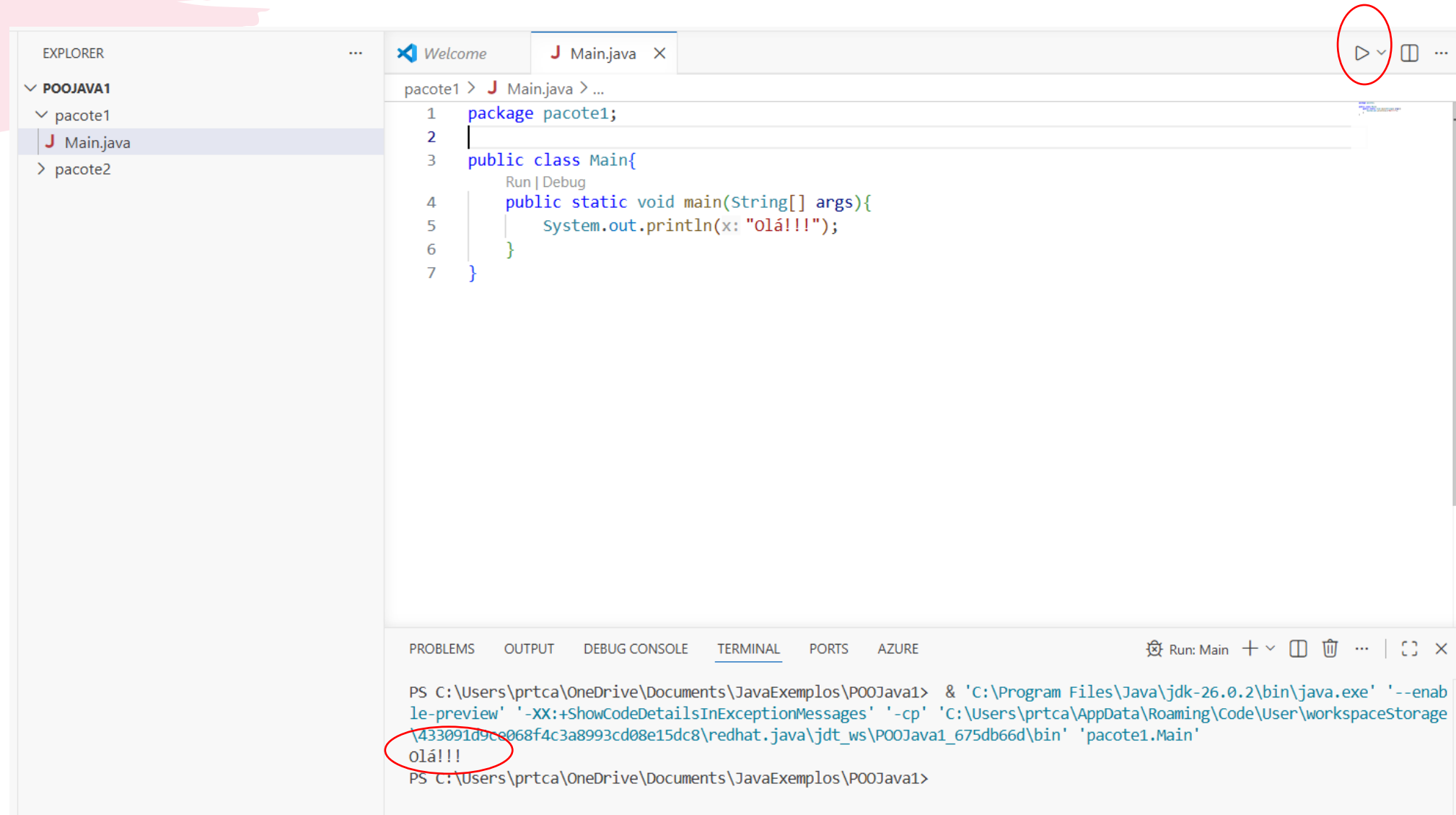
Ambiente de Programação

- Java SE Development Kit
- Variável de Ambiente: JAVA_HOME (valor correspondendo a pasta raiz do Java SE Development Kit)
- VSC
 - Extensões: Extension Pack For Java

Criar estrutura de pastas vazias



Criar classe (“Molde”) Main e executar



Classe Marca

```
pacote1 > J Marca.java > Marca > nome

1 package pacote1; // caminho até a classe
2
3 class Marca { // acessível apenas por outras classes do mesmo pacote (pasta)
4     // Atributos (dados)
5     private final int id; // final - atribuir valor única vez
6     private String nome; // private - acessível apenas por código da classe
7
8     // Métodos (operações)
9     // Construtor sem parâmetros - função executada na criação do objeto
10    Marca(){ // acessível apenas por outras classes do mesmo pacote (pasta)
11        this.id=0;
12        this.nome = "";
13    }
14 }
```

Exercício

```
EXPLORER  ...
└─ POOJAVA1
   └─ pacote1
      ├── Main.java
      └── Marca.java 2
         > pacote2

Welcome  Main.java  X  Marca.java 2

pacote1 > Main.java > Main > main(String[])
1  package pacote1; // caminho até a classe
2
3  public class Main{
   Run | Debug
4      public static void main(String[] args){
5          // Cria (instancia) um objeto do tipo Marca
6          // Funciona pois as classes estão na mesma pasta (package)
7          Marca m = new Marca();
8          Marca p = new Marca();
9
10         // Exercício -
11         // O que os prints abaixo geram?
12         // De onde vêm os métodos toString, getClass, equals e hashCode?
13         System.out.println( m );
14         System.out.println( m.toString() );
15         System.out.println( m.getClass() );
16         System.out.println( m.equals(p) );
17         System.out.println( m.hashCode() );
18     }
19 }
```

Marca – toString()

```
class Marca { // acessível apenas por outras classes do mesmo pacote (pasta)
    // Atributos (dados)
    private final int id; // final - atribuir valor única vez
    private String nome; // private - acessível apenas por código da classe

    // Métodos (operações)
    // Construtor sem parâmetros - função executada na criação do objeto
    Marca(){ // acessível apenas por outras classes do mesmo pacote (pasta)
        this.id=0;
        this.nome = "";
    }

    @Override // gera nova versão do método, sobrescrendo o herdado de java.lang.Object
    public String toString(){
        return String.format(format: "Marca[ID: %d, NOME: %s]",this.id,this.nome);
    }
}
```

equals e hashCode

```
22  @Override // Dois objetos devem ser considerados iguais de tiverem o mesmo id
23  public boolean equals(Object o) {
24      if (this == o) return true; // Mesmo endereço de memória - this corresponde ao objeto executor da operação
25      if (o == null || this.getClass() != o.getClass()) return false; // Tipos diferentes ou nulo
26      Marca que = (Marca) o;
27      return Objects.equals(id, que.id); // Compara apenas o id
28  }
29
30  @Override
31  public int hashCode() {
32      return Objects.hash(id); // Gera hash derivado unicamente do ID
33  }
```

- Necessário: import java.util.Objects;
- **IMPORTANTE:** é imprescindível que equals e hashCode sejam baseados nos mesmos dados

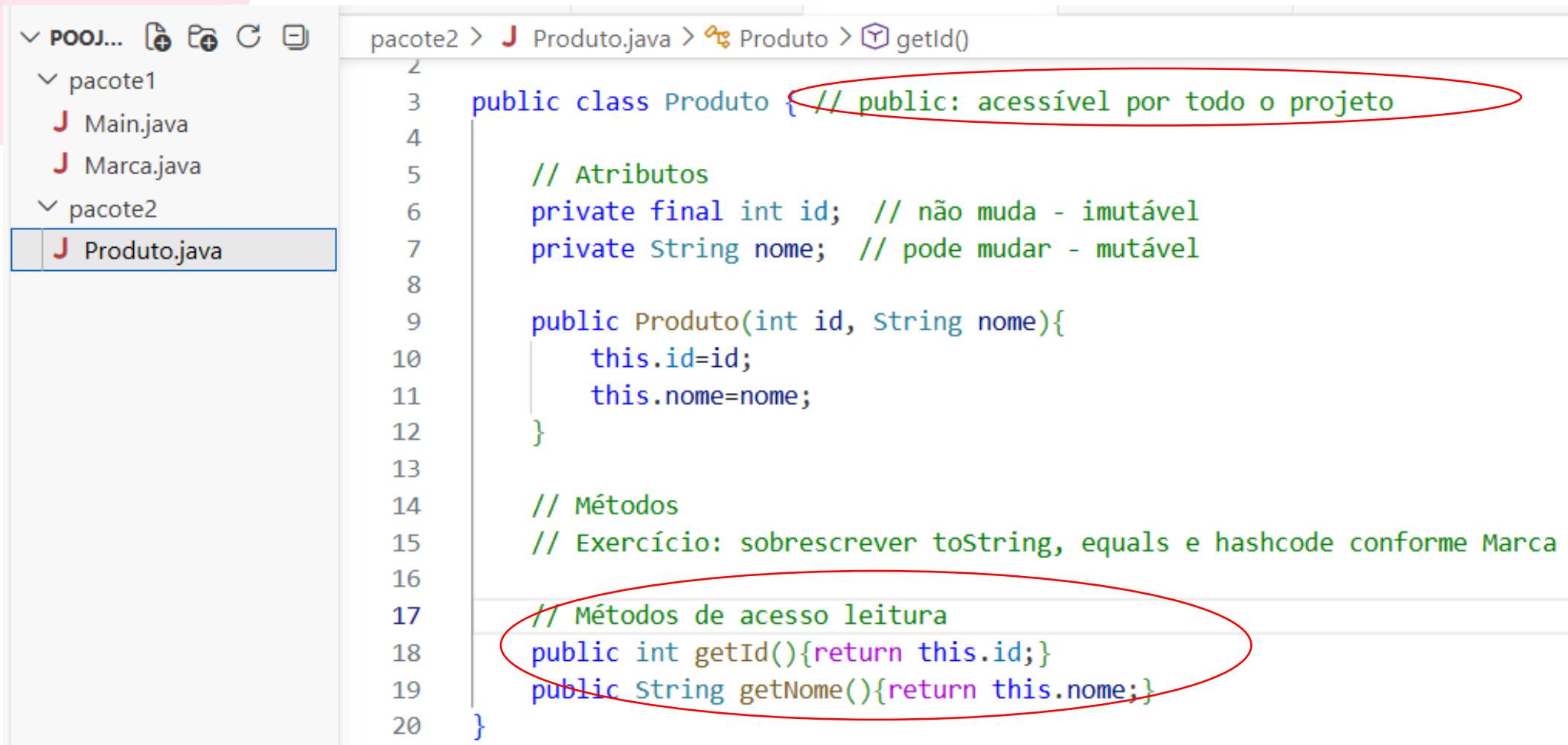
Sobrecarga do construtor

```
16
17 // Construtor com parâmetros
18  // SOBRECARGA: dois métodos com mesmo nome, mas com lista de parâmetros diferente.
19 Marca (int id, String nome){
20     this.id=id;
21     this.nome=nome;
22 }
```

Refazer testes em Main

```
1  package pacote1; // caminho até a classe
2
3  public class Main{
    Run | Debug
4      public static void main(String[] args){
5          // Cria (instancia) um objeto do tipo Marca
6          // Funciona pois as classes estão na mesma pasta (package)
7          Marca m = new Marca(id: 10,nome: "Ford");
8          Marca p = new Marca(id: 10,nome: "VW");
9
10         // Exercício -
11         // O que os prints abaixo geram?
12         // De onde vêm os métodos toString, getClass, equals e hashCode?
13         System.out.println( m );
14         System.out.println( m.toString() );
15         System.out.println( m.getClass() );
16         System.out.println( m.equals(p) ); // IMPORTANTE: resulta true pois mesmo id
17         System.out.println( m.hashCode() );
18     }
19 }
```

Classe Produto – Acesso Public



```
pacote2 > J Produto.java > Produto > getId()
2
3 public class Produto { // public: acessível por todo o projeto
4
5     // Atributos
6     private final int id; // não muda - imutável
7     private String nome; // pode mudar - mutável
8
9     public Produto(int id, String nome){
10         this.id=id;
11         this.nome=nome;
12     }
13
14     // Métodos
15     // Exercício: sobrescrever toString, equals e hashCode conforme Marca
16
17     // Métodos de acesso leitura
18     public int getId(){return this.id;}
19     public String getNome(){return this.nome;}
20 }
```

```
pacote2.Produto p1 = new pacote2.Produto(id: 100,nome: "Lápis");
System.out.println( p1.getId() + " " + p1.getNome() );
```

Teste em
Main.java

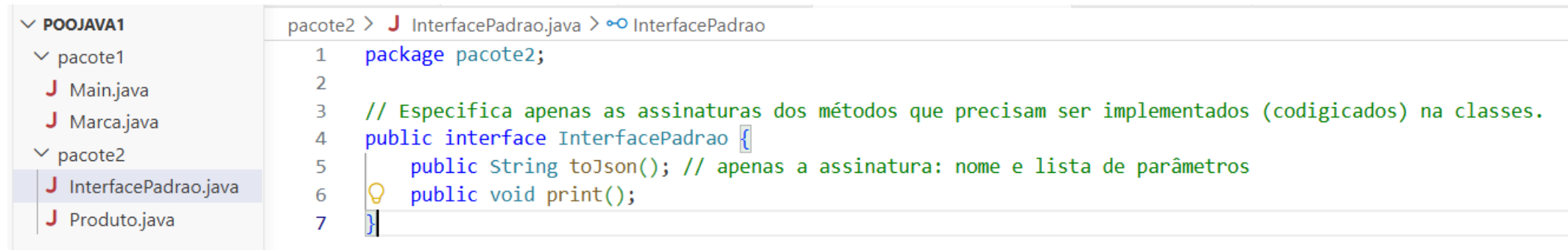
Tornar Produto Mutável e Seguro quanto a dados inválidos

```
--
11 // Torna construtos mais seguro com validação do dados recebidos nos parâmetros
12 public Produto(int id, String nome){
13     this.id = Objects.requireNonNull(id, message: "O ID não pode ser nulo."); // levanta erro se NULL
14     this.nome = validarEFormatarmNome(nome);
15 }
16
17 // private - este método não está na INTERFACE dos objetos Produto, não acessível por código fora da classe.
18 // ENCAPSULAMENTO.
19 private String validarEFormatarmNome(String nome) {
20     if (nome == null || nome.isBlank()) {
21         throw new IllegalArgumentException(s: "O nome é obrigatório.");
22     }
23     return nome.trim(); // remove brandos
24 }
25
26 // Método de acesso para mudança do nome. Obs.: id é imutável
27 public void setNome(String nome){this.nome = validarEFormatarmNome(nome);}
```

Como padronizar interfaces em classes

Supondo que se queira que todas as classes do projeto possuam os alguns métodos públicos (mesma interface)!

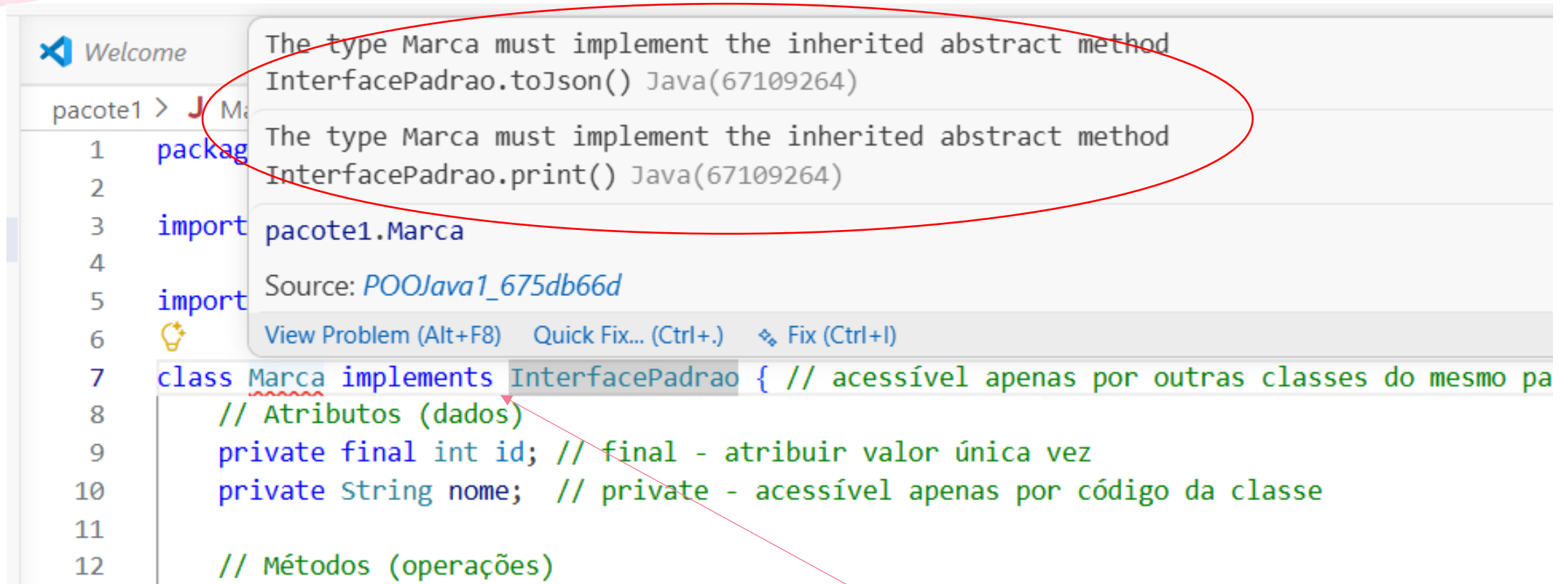
Por exemplo, os métodos “toJson()” e “print” devem ser obrigatórios em todas as classes.



The screenshot shows an IDE with a project named 'POOJAVA1'. The project structure on the left includes two packages: 'pacote1' (containing 'Main.java' and 'Marca.java') and 'pacote2' (containing 'InterfacePadrao.java' and 'Produto.java'). The 'InterfacePadrao.java' file is selected. The main editor displays the code for 'InterfacePadrao' in the 'pacote2' package. The code defines the package, a comment, and the interface with two methods: 'toJson()' and 'print()'.

```
pacote2 > J InterfacePadrao.java > InterfacePadrao
1 package pacote2;
2
3 // Especifica apenas as assinaturas dos métodos que precisam ser implementados (codificados) na classes.
4 public interface InterfacePadrao {
5     public String toJson(); // apenas a assinatura: nome e lista de parâmetros
6     public void print();
7 }
```

Classe implementa Interface



```
1 package pacote1;
2
3 import java.util.*;
4
5 import InterfacePadrao;
6
7 class Marca implements InterfacePadrao { // acessível apenas por outras classes do mesmo pa
8     // Atributos (dados)
9     private final int id; // final - atribuir valor única vez
10    private String nome; // private - acessível apenas por código da classe
11
12    // Métodos (operações)
```

Classe "Marca" precisa ter os métodos definidos na Interface. Projeto não compilará até que o métodos sejam criados na classe.

Interface implementada na Classe

```
7  class Marca implements InterfacePadrao { // acessível apenas por outras classes
8      // Atributos (dados)
9      private final int id; // final - atribuir valor única vez
10     private String nome; // private - acessível apenas por código da classe
11
12     // Métodos (operações)
13     public String toJson(){
14         return String.format(format: "{id:%d,nome:\"%s\"}",this.id,this.nome);
15     }
16
17     public void print(){
18         System.out.println( this.toString() );
19     }
20 }
```

Exercício: implementar métodos da InterfacePadrao em Produto

```
import java.util.Objects;
```



```
public class Produto implements InterfacePadrao { // public: acessível por todo o p
```

```
// Atribui
```

```
private f
```

```
private s
```

```
// Torna
```

```
public Pr
```

```
    this.
```

```
    this.
```

```
}
```

The type Produto must implement the inherited abstract method
InterfacePadrao.toJson() Java(67109264)

The type Produto must implement the inherited abstract method
InterfacePadrao.print() Java(67109264)

pacote2.Produto

Source: POOJava1_675db66d

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#) [Fix \(Ctrl+I\)](#)

Usando a interface

```
25  
26 InterfacePadrao marca = new Marca(id: 20, nome: "GM");  
27 InterfacePadrao produto = new Produto(id: 30, nome: "Caneta");  
28 InterfacePadrao[] vetor = new InterfacePadrao[] { marca, produto };  
29 System.out.println( vetor[1].toJson() ); // usa toJson() da classe Produto
```

Objetos diferentes armazenados
no mesmo vetor

```
6 public class Main{  
7  
8     public void printMain ( InterfacePadrao x ) {  
9         System.out.println( x.toJson() );  
10    }  
11
```

Pode passar um objeto Marca ou Produto neste parâmetro, pois ambas as classes implementam a InterfacePadrao.

Ambas as classes implementam método toJson() definido na InterfacePadrao.